PROJETO 8JOBS

Roteiro de apresentação

Guia de fala e demonstração: funcionalidades, arquitetura, código, banco de dados e testes.

Duração sugerida 18 a 22 minutos	Ordem Produto, código, dados, bibliotecas e testes
Frontend http://localhost:5173	API Gateway http://localhost:3000

Objetivo
Apresentar o 8Jobs como um sistema completo: a pessoa usa a interface, o frontend conversa com o gateway, os microserviços aplicam as regras e os dados são persistidos no MongoDB. Jest e Cypress ficam no encerramento.

Conteúdo baseado na documentação e no código da branch main do repositório 8Jobs.

1. Abertura e objetivo | 1 min

> “O 8Jobs é uma plataforma de vagas de emprego. Empresas podem publicar oportunidades e acompanhar candidaturas. Candidatos podem criar uma conta, procurar vagas e se candidatar. O projeto usa React no frontend, Node.js e Express no backend e MongoDB para persistência dos dados.”

> “Além de demonstrar as funcionalidades, vou explicar como o frontend se comunica com a API, como os microserviços são organizados e como os dados são armazenados.”

2. Arquitetura | 2 min

Abra o diagrama do README ou desenhe o fluxo:

```
Navegador
|
Frontend React (Vite)
| HTTP / JSON
API Gateway (:3000)
|
Empresas (:3001) | Vagas (:3002) | Candidatos (:3003) | Candidaturas (:3004)
|
MongoDB: um banco por serviço + banco de sessões
```

> “O frontend não acessa diretamente o MongoDB. Ele envia requisições HTTP ao API Gateway, que encaminha cada pedido ao microserviço responsável.”

> “Cada microserviço cuida das próprias regras e dos próprios dados. Quando um serviço precisa de informação de outro, consulta a API desse serviço.”

Exemplo: antes de gravar uma candidatura, o serviço de candidaturas consulta as APIs de candidatos e vagas. Confirma que os dois registros existem e que a vaga está aberta.

3. Demonstração das funções | 5 min

3.1 Vagas públicas

Abra http://localhost:5173. Mostre a página inicial, a lista de vagas e os detalhes de uma vaga.

> “O visitante pode consultar as oportunidades e abrir os detalhes. A tela busca os dados pela API, passando pelo gateway.”

3.2 Cadastro e login

Mostre as telas de cadastro de candidato e empresa, depois a página de login. Explique que são perfis diferentes e que o login mantém uma sessão.

> “A senha não é salva como texto puro: o backend armazena um hash. Depois do login, a sessão identifica o usuário sem expor a senha.”

3.3 Ações por tipo de conta

- Empresa autenticada: publica e administra vagas próprias, consulta candidatos e acompanha candidaturas relacionadas às suas vagas.
- Candidato autenticado: consulta vagas e pode enviar candidatura para vaga aberta.
- Rotas protegidas: a aplicação verifica autenticação e perfil antes de mostrar determinadas páginas.

> “A interface orienta a navegação, e o gateway também valida as permissões no servidor.”

Dica: use uma conta candidata para enviar candidatura a uma vaga aberta e depois entre com a empresa dona dessa vaga. Evite repetir uma candidatura já existente nos dados de exemplo.

4. Comunicação do frontend | 2 min

Abra frontend/src/services/api.js.

> “Este arquivo centraliza as chamadas HTTP. As páginas usam funções compartilhadas para enviar pedidos ao gateway, incluir o cookie da sessão e interpretar respostas e erros.”

- fetch envia a requisição HTTP.
- credentials: include permite enviar o cookie de sessão junto com a chamada.
- enviarJson facilita o envio de dados JSON em cadastros e atualizações.
- VITE_API_URL permite configurar o endereço da API; o padrão local aponta para o gateway.

Depois, mostre frontend/src/App.jsx: registra rotas de navegação e indica quais páginas exigem conta de empresa ou de candidato.

5. Gateway e microserviços | 3 min

Abra gateway/app.js. Aponte as URLs internas dos serviços e o middleware que encaminha os pedidos recebidos em /api.

> “O gateway é a entrada pública do backend. Ele recebe as chamadas, identifica o recurso e encaminha o pedido ao microserviço correspondente.”

- Rotas de autenticação: login, consulta da sessão e logout.
- Verificação de sessão, perfil e propriedade dos dados antes de encaminhar certas operações.
- Tratamento de indisponibilidade e timeout de serviços.

Mostre também services/candidaturas/src/controllers/candidaturaControllers.js.

> “O controller recebe a requisição, aplica as regras do serviço e usa o model para consultar ou alterar dados. A resposta usa códigos HTTP: 201 para criação, 400 para dados inválidos e 404 quando o registro não existe.”

Separação do código: routes mapeia URL e método HTTP; controllers implementa o fluxo; models descreve os dados; clients consulta outros microserviços por HTTP.

6. Banco de dados | 2 min

Abra o MongoDB Compass e mostre a conexão local e os bancos, se já estiverem criados.

> “O projeto usa um banco para cada microserviço: empresas, vagas, candidatos e candidaturas. O gateway mantém um banco adicional para sessões.”

- 8jobs_empresas - coleção empresas
- 8jobs_vagas - coleção vagas
- 8jobs_candidatos - coleção candidatos
- 8jobs_candidaturas - coleção candidaturas
- 8jobs_sessions - sessões do gateway

> “Uma vaga guarda o ID da empresa responsável. Uma candidatura guarda o ID do candidato e o ID da vaga. Como os dados pertencem a serviços diferentes, a relação usa identificadores e consultas entre APIs.”

Abra services/candidaturas/src/models/candidaturaModel.js. Mostre os campos, as validações do status e o índice único que impede a mesma pessoa de se candidatar duas vezes à mesma vaga.

Os scripts database/criar-banco.js e database/popular-banco.js preparam a estrutura e inserem dados de demonstração.

7. Bibliotecas principais | 2 min

- React: cria a interface usando componentes reutilizáveis.
- React Router: navega entre páginas sem recarregar o site inteiro.
- Vite: servidor de desenvolvimento e ferramenta de build do frontend.
- Tailwind CSS: oferece classes utilitárias para estilizar elementos diretamente no JSX, como cores, margens e espaçamentos.
- Express: cria servidores, rotas e middlewares do backend.
- Mongoose: define modelos e validações e permite acessar o MongoDB pelo Node.js.
- CORS: configura quais origens do navegador podem acessar a API. O gateway permite a origem do frontend e credenciais para a sessão.
- express-session e connect-mongo: gerenciam sessões e as armazenam no MongoDB.
- express-rate-limit: limita tentativas repetidas de login.
- dotenv: carrega configurações locais de arquivos .env.

8. Como iniciar a demonstração | 1 min

Antes da apresentação, confirme que o MongoDB Server está ligado. Na primeira configuração, instale as dependências e inicie os serviços na raiz:

```
npm run setup
npm run dev
```

> “O comando npm run dev inicia o gateway, os quatro microserviços e o frontend. O terminal identifica cada processo. Para encerrar tudo, pressiono Ctrl+C.”

Acesse o frontend em http://localhost:5173. Se o Vite escolher outra porta, use o endereço do terminal. O gateway da API fica em http://localhost:3000.

Opcional: criar estrutura e dados demonstrativos, se o MongoDB estiver configurado:

```
npm run db:criar
npm run db:popular
```

> **Atenção: npm run db:popular -- --zerar apaga os registros dos bancos antes de inserir exemplos. Não use se houver dados que queira manter.**

9. Testes: Jest e Cypress | 2 min

Jest: testes das APIs

> “O Jest testa as APIs dos microserviços. Os testes verificam respostas de sucesso e situações de erro. A maioria simula MongoDB e chamadas entre serviços, então não exige iniciar o banco nem todos os servidores.”

Na raiz do projeto, execute:

```
npm test -- --verbose
```

Mostre no terminal os nomes dos testes e o resumo das suítes aprovadas. A suíte de integração com MongoDB real é separada e exige configurar MONGODB_TEST_URI.

Cypress: testes pelo navegador

> “O Cypress testa os fluxos da interface pelo navegador. Os testes E2E usam uma API simulada, então não alteram os dados reais do backend.”

Inicie o frontend em um terminal e abra o Cypress em outro:

```
npm run dev:frontend
npm run cy:open
```

Jest verifica as APIs; Cypress verifica os caminhos do usuário pela interface.

10. Encerramento | 30 s

> “Em resumo, o 8Jobs combina uma interface React, um gateway que centraliza chamadas e controla sessões e permissões, quatro microserviços com responsabilidades separadas e bancos MongoDB próprios. O projeto também conta com testes automatizados para APIs e fluxos da interface.”

Agradeça e abra para perguntas.

Checklist antes de apresentar

- MongoDB Server ligado e variáveis .env configuradas.
- Dependências instaladas; confirme npm run setup com antecedência.
- Projeto iniciado com npm run dev; frontend aberto no endereço informado pelo Vite.
- MongoDB Compass aberto nos bancos 8jobs_*, se for mostrar os dados.
- Conta de empresa e conta de candidato disponíveis para demonstrar os dois perfis.
- Escolha uma vaga aberta sem candidatura duplicada para a conta de demonstração.
- Terminal na raiz para executar npm test no encerramento.

Referências do projeto

Documentação principal: README.md da branch main.

Arquivos para abrir: frontend/src/App.jsx; frontend/src/services/api.js; gateway/app.js; services/candidaturas/src/controllers/candidaturaControllers.js; services/candidaturas/src/models/candidaturaModel.js; database/criar-banco.js; database/popular-banco.js.